

Setu Payment Lifecycle & Reconciliation Service

Executive Summary :

In modern fintech ecosystems, payment processing is rarely a single synchronous action. A single customer transaction involves multiple asynchronous hops between banks, gateways, and merchants. This project serves as the centralized source of truth for these events. It exposes a RESTful API to safely ingest continuous streams of payment data, reconstructs the journey of every transaction, and surfaces real-time financial analytics and anomalies.

Problem Statement :

Payment gateways and financial aggregators face several critical engineering challenges when handling distributed payment events at scale:

1. **Network Unreliability & Duplicate Data:** Network timeouts frequently cause payment partners to retry sending the same event webhook multiple times. Without strict idempotency, a system might process the same payment twice, leading to catastrophic double-credits.
2. **Fragmented Transaction States:** Events like `payment_initiated`, `payment_processed`, and `settled` arrive at different times (often days apart). Tracking the exact current state of a payment requires intelligently linking these fragmented events into a single, cohesive state machine.
3. **The "Black Hole" of Settlement (Reconciliation):** The most critical challenge for merchants is ensuring that a payment marked as "Successful" by the gateway actually results in money being deposited into their bank account (Settled). Manually finding transactions where money is "stuck" or where fraudulent settlements occur is virtually impossible at high volumes.

The Objective:

The core requirement of this project was to build a backend service using **Python with FastAPI, backed by an SQL database**. The system is required to support payment event **ingestion, transaction retrieval, reconciliation** reporting in a way that is production-minded, deployable, and highly efficient.

The Solution:

This service was built specifically to solve these challenges. It guarantees safe, idempotent event ingestion, automatically derives the master state of every transaction, and features a purpose-built reconciliation engine that flags impossible or stuck financial states (like processed payments that were never settled) before they impact merchant payouts.

System Architecture & Tech Stack

To ensure the system is "production-minded, deployable, and efficient," the architecture was designed around modern, lightweight, and scalable technologies, prioritizing speed and developer experience.

Technology Stack :

Application	Technology Used	Why Chosen
Backend Framework	FastAPI (Python)	High Performance (ASGI), Data Validation on Pydantic, OpenAPI schema, ASYNC support
Database	Supabase (PostgreSQL via SQLAlchemy)	Chosen because serverless deployments (like Vercel) require a remote, persistent database
Deployment & Hosting	Vercel	The API is deployed as a serverless function on Vercel
Documentation & UI	HTML5, Vanilla CSS, and ReDoc	Designed to give any layman user to read the document and understand

Core Architectural Principles:

The system was engineered using four important design principles:

A. Event-Sourcing & State Machine Derivation :

Instead of directly mutating a transaction's status, the system uses an event-driven model. The events table is an immutable ledger of everything that has happened.

When a new event (e.g., payment_processed) arrives, it is appended to the ledger.

The system then immediately evaluates the transaction's history and calculates its derived state (updating the payment_status, settlement_status, and overall status in the transactions table).

Benefit: We never lose historical context, and we can perfectly reconstruct the timeline of any transaction.

B. Idempotency Handling (Duplicate Handling) : Network failures cause retries. If the payment gateway sends the exact same payment_initiated event three times due to a timeout, processing it three times would corrupt the financial data.

The system enforces strict idempotency using the event_id as a unique primary key constraint.

If a duplicate event_id is ingested, the database catches the integrity error. The API gracefully swallows the error and returns a 200 OK with an ingestion_status of already_processed.

Benefit: The API is completely safe to retry, fulfilling the highest standard of fintech reliability.

C. Database Schema Design (Supabase / PostgreSQL)

The PostgreSQL database (hosted on Supabase) is designed for both high-throughput writes (event ingestion) and complex analytical reads (reconciliation). It consists of two primary, heavily indexed tables:

transactions Table: Stores the aggregated, top-level state of a payment journey (Merchant ID, Total Amount, Derived Status). Optimized for fast UI dashboard queries and filtering.

events Table: Stores the immutable, granular webhook payloads. Linked to the transactions table via a Foreign Key (transaction_id) with cascading constraints to ensure relational integrity.

D. The Reconciliation Engine (Native Postgres Aggregation)

Rather than pulling raw data into Python memory to calculate financial totals (which would crash the server at high volumes), the reconciliation endpoints push the heavy lifting down directly to the Supabase PostgreSQL engine.

The /reconciliation/summary endpoint utilizes optimized Postgres GROUP BY and CASE WHEN SQL statements to aggregate transaction counts, calculate settlement rates, and sum monetary totals across millions of rows in milliseconds.

Benefit: Leveraging Supabase's managed Postgres instance ensures that these heavy analytical queries execute blazing fast without bogging down the FastAPI application layer.

Core Features & API Endpoints :

The API is structured into three logical domains: Data Ingestion, Data Retrieval, and Financial Reconciliation. Every list-based endpoint supports comprehensive pagination and dynamic filtering.

Base URL : <https://setu-payment-service.vercel.app>

Method	Endpoint	Domain	Purpose	Key Features & Filters
POST	/events	Ingestion	Ingest a raw payment lifecycle event and update the parent transaction's state.	Idempotent (Duplicate event_ids are safely ignored and return already_processed).
GET	/events	Retrieval	Fetch a paginated ledger of all raw events ever ingested.	Filter by: event_type, transaction_id, merchant_id.
GET	/transactions	Retrieval	Fetch a paginated list of top-level transactions and their master derived status.	Filter by: merchant_id, status, date_from, date_to. Sortable by amount or date.
GET	/transactions/{id}	Retrieval	Fetch granular details of a single transaction.	Returns the complete chronological event audit trail for the specific transaction.
GET	/reconciliation/summary	Reconciliation	Generate a real-time, SQL-aggregated financial summary.	Returns data aggregated three ways: by merchant (settlement rates), by status, and by date.
GET	/reconciliation/discrepancies	Reconciliation	The anomaly engine. Flags transactions in stuck or illogical financial states.	Detects: stale_initiated, processed_not_settled, and settled_after_failure (fraud detection).

Applicable States and Supported Filter Types/Query Params :

Method	Endpoint	Supported Filters (Query Params)	Applicable States & Types
POST	/events	(Body parameters only)	Event Types: payment_initiated, payment_processed, payment_failed, settled
GET	/events	transaction_id, merchant_id, event_type, page, page_size, sort_order	Event Types: payment_initiated, payment_processed, payment_failed, settled
GET	/transactions	merchant_id, status, date_from, date_to, page, page_size, sort_by, sort_order	Master Statuses: initiated, processed, failed, settled
GET	/transactions/{id}	(Path parameter only)	Derived Statuses: payment_status, settlement_status, status
GET	/reconciliation/summary	merchant_id, date_from, date_to	Aggregated Statuses: initiated, processed, failed, settled
GET	/reconciliation/discrepancies	merchant_id, discrepancy_type	Discrepancy Types: stale_initiated, processed_not_settled, settled_after_failure

Developer Experience (DX), UI & Documentation :

A backend service is only as good as its documentation. To ensure this API is instantly usable by frontend engineers and integration partners, a significant focus was placed on Developer Experience (DX) and visual tooling.

The Branded Developer Portal (/redoc)

URL: <https://setu-payment-service.vercel.app/redoc>

Instead of relying solely on default API schemas, the project features a custom, visually appealing documentation portal built from the ground up. This contains

Fintech 101 Glossary: Clear definitions of event_id, transaction_id, and Idempotency for non-technical stakeholders.

State Machine Matrix: A visual breakdown of how events progress through the system.

Endpoint Cards: Beautifully styled reference cards for all endpoints, explaining their exact business purpose rather than just their technical schema.

Interactive Swagger UI (/docs)

URL: <https://setu-payment-service.vercel.app/docs>

FastAPI automatically generates an interactive OpenAPI (Swagger) interface. This allows developers to execute live requests, test filtering parameters, and view exact request/response schemas directly from their browser without writing any code.

Discrepancy Detection Logic :

The system does not just look at individual events in isolation; it derives a master state for every transaction by rolling up its entire event history. The anomaly detection engine then evaluates this derived state against specific business rules to flag any transaction that enters an illogical or "stuck" financial state.

Discrepancy Type	Detection Condition (The Logic)	What It Means	Real-World Impact
settled_after_failure	payment_status == "failed" AND settlement_status == "settled"	The customer's payment was marked as failed, but a settlement event arrived depositing money to the merchant anyway.	High Severity. This is an impossible state that indicates a severe system desync, potential fraud, or a double-credit bug.
processed_not_settled	payment_status == "processed" AND settlement_status == "pending"	The customer's payment successfully went through, but the settlement event has not arrived.	Medium Severity. The merchant is owed money that is currently "stuck" somewhere in the banking network and hasn't been paid out.
stale_initiated	status == "initiated" AND Only 1 event exists in history	A payment was initiated by a customer, but no follow-up success or failure event was ever received.	Low/Medium Severity. Could be normal user drop-off (closing the browser), or it could mean webhook messages from the payment partner are dropping.